

TABELLE JOINED

- Il concetto di **tabella joined** compare con SQL2 per specificare un'operazione di **JOIN** nella clausola **FROM**.

- Possono essere usati i seguenti tipi di join:

- NATURAL JOIN**
- INNER JOIN**
- LEFT OUTER JOIN / RIGHT OUTER JOIN**
- FULL OUTER JOIN**

in SQL di default

Sintassi :
Date le relazioni $R(A_1, A_2, \dots, A_n)$ e $S(B_1, B_2, \dots, B_m)$
Scriveremo $R \bowtie_{\langle \text{join_condition} \rangle} S$

- Il risultato della join ha $n+m$ attributi:
 $Q(A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m)$

AND: Congiunzione logica

- La condizione della join è della forma:
 $\langle \text{condition} \rangle \text{ AND } \langle \text{condition} \rangle \text{ AND } \dots \text{ AND } \langle \text{condition} \rangle$
dove ogni condizione è della forma $A_i \theta B_j$
con $A_i \in R, B_j \in S, \text{dom}(A_i) = \text{dom}(B_j)$ e $\theta \in \{=, \neq, <, >, \leq, \geq\}$

- Tuple con attributi di join **null** non appaiono nel risultato.

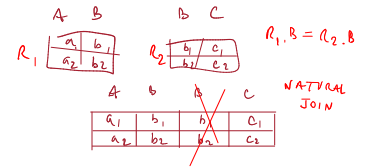


TABELLE JOINED - ESEMPIO

- Trovare il nome e l'indirizzo di ogni impiegato che lavora per il Dipartimento **'Research'**:

```
SELECT FNAME, LNAME, ADDRESS
FROM (EMPLOYEE JOIN DEPARTMENT ON
DNO=DNUMBER)
WHERE DNAME='Research';
```

EQUI JOIN

AGGREGAZIONE E RAGGRUPPAMENTO

- Le funzioni di aggregazione e di raggruppamento sono diffusissime nella gestione di basi di dati. SQL incorpora le seguenti funzioni:

- COUNT**: conteggio tuple.
- COUNT(DISTINCT ...)**: conteggio di tuple distinte.
- SUM**: somma dei valori di un attributo in una tabella.
- MAX**: valore massimo tra gli attributi di una tabella.
- MIN**: valore minimo tra gli attributi di una tabella.
- AVG**: valore medio tra gli attributi di una tabella. — *media aritmetica*
- STD**: deviazione standard tra gli attributi di una tabella.

J

- Esempio**: Trovare la somma dei salari di tutti gli impiegati, il massimo, il minimo e la media dei salari:

```
SELECT SUM(SALARY), MAX(SALARY),
MIN(SALARY), AVG(SALARY)
FROM EMPLOYEE;
```

COUNT - ESEMPI

- Conta il numero di impiegati:

```
SELECT COUNT(*)  
FROM EMPLOYEE;
```

Conta i record di EMPLOYEE

- Restituisce il numero di tuple nel risultato della query (*):

```
SELECT COUNT(*) AS Conteggio  
FROM EMPLOYEE, DEPARTMENT  
WHERE DNO=DNUMBER AND DNAME='Research';
```

*Conta gli impiegati del
dipartimento "Research"*

- Conta il numero di valori di stipendi distinti:

```
SELECT COUNT (DISTINCT SALARY)  
FROM EMPLOYEE;
```

- Elencare il nome ed il cognome degli impiegati che hanno due o più persone a carico:

```
SELECT LNAME, FNAME  
FROM EMPLOYEE  
WHERE ( SELECT COUNT(*)  
        FROM DEPENDENT  
        WHERE SSN=ESSN ) >= 2;
```

query annidata

ORDINAMENTO DI TUPLE

- Per ordinare le tuple nel risultato della query si usa la clausola **ORDER BY**.

- **Esempio:** Ritrovare una lista di impiegati e dei progetti su cui lavorano, ordinati per dipartimento, e nell'ambito di ciascun dipartimento, alfabeticamente per cognome e nome:

```
SELECT DNAME, FNAME, LNAME, PNAME  
FROM DEPARTMENT, EMPLOYEE, WORKS_ON, PROJECT  
WHERE DNUMBER=DNO AND SSN=ESSN AND  
      PNO=PNUMBER  
ORDER BY DNAME, LNAME, FNAME;
```

- L'ordine di default è **crescente**:

- **ASC** per crescente.
- **DESC** decrescente.

- **Esempio:** per avere un ordine decrescente di dipartimento e crescente per nome e cognome:

...

```
ORDER BY DNAME DESC, LNAME ASC, FNAME ASC;
```

GROUP BY

- Raggruppiamo le tuple che hanno lo stesso valore per alcuni attributi.

Esempio:

```
SELECT DNO, COUNT(*), AVG(SALARY)
FROM EMPLOYEE
GROUP BY DNO;
```

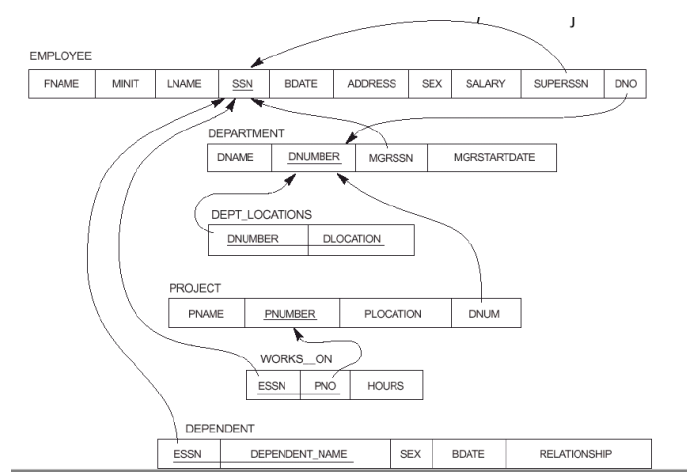
*ciò che vogliamo
proiettare / aggregare*

- Le tuple sono divise in gruppi, ogni gruppo ha lo stesso valore per DNO.
- Le funzioni COUNT e AVG sono applicate ad ogni gruppo di queste tuple.

Risultato:

DNO	COUNT(*)	AVG(SALARY)
1	4	23000
4	3	25000
3	4	22000

*Le operazioni aggregate
si applicano a ciascun
gruppo anziché a tutti
i record*



- Esempio:** Per ogni progetto, visualizzare il numero del progetto, il nome del progetto ed il numero di impiegati che lavorano su quel progetto:

```
SELECT pnumber, pname, COUNT(*)
FROM project, works_on
WHERE pnumber = pno
GROUP BY pnumber, pname;
```

- Esempio:** Per ogni progetto visualizzare il numero del progetto, il nome del progetto ed il numero di impiegati del dipartimento n.5 che lavorano su quel progetto:

```
SELECT pnumber, pname, COUNT(*)
FROM project, works_on, employee
WHERE pnumber = pno AND ssn = essn AND dno = 5
GROUP BY pnumber, pname;
```

GROUP BY (5) – USO DI HAVING

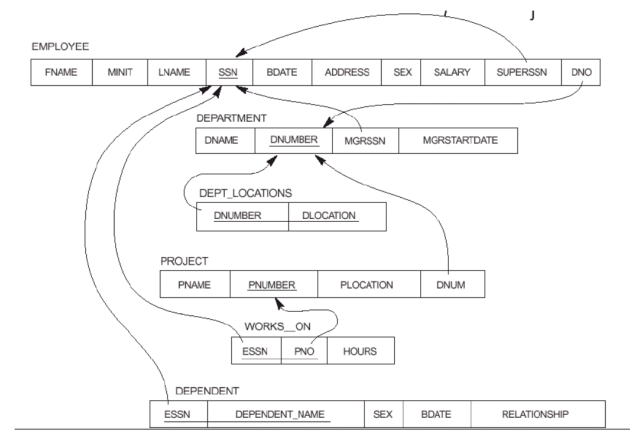
- Esempio:** Per ogni progetto su cui lavorano più di due impiegati, visualizzare il numero del progetto, il nome del progetto ed il numero di impiegati che lavorano su quel progetto:

```
SELECT pnumber, pname, COUNT(*)
FROM project, works_on
WHERE pnumber = pno
GROUP BY pnumber, pname
HAVING COUNT(*) > 2;
```

GROUP BY (6) — USO DI HAVING (2)

- **Esempio:** Determinare, per ogni dipartimento che ha più di 6 impiegati, il numero totale degli impiegati il cui stipendio è maggiore di \$40.000:

```
SELECT dname, COUNT(*)  
FROM department, employee  
WHERE dnumber = dno AND salary > 40000  
GROUP BY dname  
HAVING COUNT(*) > 6;
```



RIEPILOGO DELLE INTERROGAZIONI

```
SELECT <elenco attributi e funzioni>  
FROM <elenco delle tabelle>  
[WHERE <condizioni>]  
[GROUP BY <attributo o attributi di raggruppamento>]  
[HAVING <condizione di raggruppamento>]  
[ORDER BY <elenco attributi>]
```

```
CREATE TABLE Fornitori (
  fid INT PRIMARY KEY,
  fnome VARCHAR(50) NOT NULL,
  indirizzo VARCHAR(100)
);
```

```
CREATE TABLE Pezzi (
  pid INT PRIMARY KEY,
  pnome VARCHAR(50) NOT NULL,
  colore VARCHAR(20),
  peso DECIMAL(5,2)
);
```

```
CREATE TABLE Catalogo (
  fid INT,
  pid INT,
  costo DECIMAL(8,2),
  PRIMARY KEY (fid, pid),
  FOREIGN KEY (fid) REFERENCES Fornitori(fid),
  FOREIGN KEY (pid) REFERENCES Pezzi(pid)
);
```

Per ogni fornitore
calcolare quanti pezzi esso
fornisce e il prezzo medio

▪ **Esempio:**
SELECT DNO, **COUNT**(*), **AVG**(SALARY)
FROM EMPLOYEE
GROUP BY DNO;

la stessa me col nome
nel risultato

```
SELECT F.fnome AS Fornitore, COUNT(pid) AS numPezzi,
avg(costo) AS costoMedio
FROM Fornitori AS F JOIN Catalogo as C ON F.fid=C.fid
GROUP BY F.fnome;
```

Fornitore	numPezzi	costoMedio
1 Polito	2	0.085
2 RomaTre	3	0.09333333333333333
3 Sapienza	3	0.11
4 Tor Vergata	2	0.185

```
SELECT fid, COUNT(pid), avg(costo)
FROM Catalogo
GROUP BY fid;
```

	fid	COUNT(pid)	avg(costo)
1	1	3	0.11
2	2	2	0.185
3	3	3	0.09333333333333333
4	4	2	0.085

Calcolare il costo totale di catalogo per ogni colore di pezzo,
ordinando per costo crescente.

```
SELECT p.colore, SUM(c.costo) AS costo_totale
FROM CATALOGO c JOIN PEZZI p ON c.pid = p.pid
GROUP BY p.colore
ORDER BY costo_totale;
```

colore	costo_totale
1 blu	0.25
2 verde	0.4
3 rosso	0.5

Trovare i fornitori il cui costo medio dei pezzi sia
superiore a 0.10, usando HAVING

```
SELECT f.fid, f.fnome, AVG(c.costo) AS costo_medio
FROM FORNITORI f JOIN CATALOGO c
ON f.fid = c.fid
GROUP BY f.fid, f.fnome
HAVING AVG(c.costo) > 0.1;
```

fid	fnome	costo_medio
1	1 Sapienza	0.11
2	2 Tor Vergata	0.185

Elencare i fornitori che forniscono almeno un pezzo che non e' di colore 'rosso',
usando LEFT JOIN.

```
SELECT DISTINCT f.fid, f.fnome
FROM FORNITORI f
JOIN CATALOGO c
ON f.fid = c.fid
LEFT JOIN PEZZI p
ON c.pid = p.pid AND p.colore = 'rosso'
WHERE p.pid IS NULL;
```

Questo esempio serve per mostrare l'uso
della LEFT JOIN ma la soluzione piu' naturale
e' diversa.

```
SELECT F.fnome
FROM Fornitori AS F JOIN Catalogo AS C ON C.fid=F.fid
WHERE F.fid NOT IN
(SELECT F1.fid FROM Fornitori AS F1
JOIN Catalogo AS C1 ON C1.fid=F1.fid);
```

Conta il numero di colori distinti che ciascun fornitore offre,
includendo fornitori che non forniscono pezzi (usare RIGHT JOIN).

```
SELECT f.fid, f.fnome, COUNT(DISTINCT p.colore) AS num_colori
FROM PEZZI p
  RIGHT JOIN CATALOGO c
    ON p.pid = c.pid
  RIGHT JOIN FORNITORI f
    ON c.fid = f.fid
GROUP BY f.fid, f.fnome;
```

```
SELECT *
FROM FORNITORI f
  LEFT JOIN CATALOGO c
    ON f.fid = c.fid
  LEFT JOIN PEZZI p
    ON c.pid = p.pid AND p.colore = 'rosso';
```

*Qui compare solo
il colore 'rosso' in
virtù della condizione
di join*

	fid	fnome	indirizzo	fid	pid	costo	pid	pnome	colore	peso
1	1	Sapienza	Via Cavour, 18	1	10	0.1	10	Vite M4	rosso	0.05
2	1	Sapienza	Via Cavour, 18	1	11	0.15	NULL	NULL	NULL	NULL
3	1	Sapienza	Via Cavour, 18	1	12	0.08	12	Dado M6	rosso	0.03
4	2	Tor Vergata	Via Columbia, 2	2	10	0.12	10	Vite M4	rosso	0.05
5	2	Tor Vergata	Via Columbia, 2	2	13	0.25	NULL	NULL	NULL	NULL
6	3	RomaTre	Viale Ostiense, 159	3	11	0.14	NULL	NULL	NULL	NULL
7	3	RomaTre	Viale Ostiense, 159	3	12	0.09	12	Dado M6	rosso	0.03
8	3	RomaTre	Viale Ostiense, 159	3	14	0.05	NULL	NULL	NULL	NULL
9	4	Polito	Corso Duca degli Abruzzi, 24	4	10	0.11	10	Vite M4	rosso	0.05
10	4	Polito	Corso Duca degli Abruzzi, 24	4	14	0.06	NULL	NULL	NULL	NULL